Research Proposal Self-Review Checklist

Tick every box before you send your proposal to your adviser. Each unticked box is something a panel is likely to flag.

How to use

Run this on a complete draft, ideally a day after writing it, so you read it as a panelist would.

1. Work through each section with your draft open beside it.
2. Tick a box only if you can point to the page where it is satisfied.
3. For each unticked box, note the page and what you will change, then revise.
4. Do the alignment check last; it catches problems no single section shows.

The guides use computer systems examples. If your area differs, swap in its own measures (e.g., accuracy for ML, task success for HCI).

1. Title

- [ ] It names the system or component, the problem, and the context (platform, workload, environment).
- [ ] It shows the contribution (e.g., "Optimizing…", "A Scheduler for…", "Detecting…").
- [ ] It contains the keywords someone would type to find this work.
- [ ] It is roughly 8–15 words.
- [ ] It has no filler such as "A Study on", "Towards" or "An Analysis of" (unless your template requires it).
- [ ] It does not overpromise ("Revolutionizing…") or rely on a catchy but vague phrase.
- [ ] Its scope matches the problem statement, aim and objectives: no broader, no narrower.
- [ ] It follows your field's or venue's style (e.g., "Optimizing X for Y", "X: A System for Z").

2. Research gap

- [ ] I can state the gap in one sentence as a specific unanswered question, unaddressed problem or missing approach, not just a topic.

- [ ] The gap is backed by evidence I cite: limitations or "future work" in papers, unrealistic assumptions, weak evaluations, issue trackers, industry reports, or my own benchmark or prototype.
- [ ] I searched recent work (last 3–5 years) and found nothing that already closes the gap.
- [ ] I explain why closing the gap matters, and to whom.
- [ ] I can do it with the tools, data, hardware and time I actually have.
- [ ] It is new but tractable: not a reimplementation, not an open-ended grand challenge.
- [ ] My adviser or peers have seen the gap and agree it is worth pursuing.

3. Problem statement

- [ ] It names the core limitation of existing systems or approaches.
- [ ] It gives enough context for a non-expert to follow.
- [ ] It is scoped to a specific domain, platform or workload.
- [ ] It states the consequences (e.g., poor performance, wasted resources, limited scalability) and who is affected.
- [ ] It says why existing work has not solved it, citing specific shortcomings.
- [ ] It says why now: a current trend makes the problem pressing.
- [ ] The core statement fits in 1–2 formal sentences.
- [ ] Words like "better", "improved" or "efficient" say better at what.
- [ ] It frames the need without describing my solution.
- [ ] It follows directly from the gap and leads naturally into the aim.

4. Aim and objectives

Aim (general objective)

- [ ] It is a single statement of 1–2 sentences.
- [ ] It names the specific problem and targets the gap.
- [ ] It states the intended outcome (e.g., higher throughput, better utilization, fairness, accuracy).
- [ ] A reader outside my subfield can understand it.

Specific objectives

- [ ] There are about 3–5 objectives.

- [ ] Each starts with an action verb (design, implement, evaluate, compare, assess), not "understand", "learn" or "explore".
- [ ] Each is SMART: specific, measurable, achievable, relevant and time-bound (a timeline elsewhere counts).
- [ ] They cover both building (design, implement) and evaluating (performance, scalability, robustness).
- [ ] The evaluation objectives name the metrics and what I compare against.
- [ ] Completing every objective would achieve the aim, and no objective falls outside it.

5. Methods

- [ ] They cover the whole pipeline: design, implementation, experimental setup, evaluation and validation.
- [ ] They name concrete tools: languages, frameworks, platforms, datasets and benchmarks.
- [ ] They describe the setup: hardware or testbed, software stack, workloads or datasets.
- [ ] They explain how the design addresses the specific limitation in the gap.
- [ ] They reuse existing tools and datasets where possible, and every resource is one I can actually get.
- [ ] They include a comparison against existing or state-of-the-art solutions.
- [ ] Every objective maps to at least one method step.

6. Evaluation and validation

- [ ] Every metric is defined, has a unit, and is tied to an objective.
- [ ] The metrics cover what matters for my problem: performance, efficiency, scalability and robustness, or my field's equivalents.
- [ ] I compare against both the common default and at least one state-of-the-art alternative, and say why I chose each.
- [ ] I use standard benchmarks or realistic datasets, in a realistic environment.
- [ ] I test varied conditions: small vs. large scale, low vs. high load, and failure or edge cases.
- [ ] Hardware and software versions are fixed, and I state the control variables.
- [ ] I say how I will confirm correctness (logs, tracing, test cases, ground truth).

- [ ] I stress or fault-test the system and state what degradation is acceptable.
- [ ] I plan multiple runs per scenario and report mean or median with standard deviation.
- [ ] I plan significance tests (e.g., t-test) for any claim that one system beats another.
- [ ] I describe the charts and tables I will use to compare against baselines.
- [ ] I plan to share code, configurations and data or traces, and to document the setup.
- [ ] I state what the evaluation will not cover and how that limits my conclusions.

7. Review of related literature

- [ ] It is organized by themes, not as a paper-by-paper or chronological list.
- [ ] It includes seminal works and recent ones (last 3–5 years).
- [ ] For each key work I give its contribution, its strength and its limitation.
- [ ] Each limitation is explicitly linked to my gap.
- [ ] Sources are mostly peer-reviewed venues and journals, plus official documentation where relevant; I don't rely on blogs or Wikipedia.
- [ ] It stays focused: about 1–2 paragraphs per theme and a core set of key works (around 6–10 for a proposal).
- [ ] Transitions connect the themes and build toward the gap.
- [ ] It ends with a synthesis paragraph that sums up the shortcomings and positions my work as the answer.
- [ ] Technical terms are defined or clear from context.
- [ ] Citations follow one style (e.g., IEEE or ACM), entries are complete with DOIs or URLs, and every in-text citation is in the reference list and vice versa.

8. Alignment check

One story should run through the whole proposal: gap → problem statement → aim → objectives → methods → metrics → title. Fill in one row per objective; an empty cell is a break in that story.

Objective	Method step(s)	Metric(s) or validation	Baseline(s)	Gap it addresses
1.				
2.				

Objective	Method step(s)	Metric(s) or validation	Baseline(s)	Gap it addresses
3.				
4.				

- [ ] Every objective has a method, a metric and a baseline.
- [ ] Every method step and metric serves at least one objective.
- [ ] The limitations in my literature review are the same gap my problem statement names.
- [ ] My title and aim are no broader or narrower than my objectives.
- [ ] My scope and limitations section does not contradict any objective.

9. Before you submit

- [ ] Every section listed above is present, whatever my template calls it.
- [ ] Terms are used consistently throughout (the same name for the same system, metric or dataset).
- [ ] Figures and tables are numbered, captioned and referred to in the text.
- [ ] I can answer, in one sentence each: What is the gap? Why does it matter now? What will I build? How will I know it works?
- [ ] Someone else (adviser, peer or classmate) has read the full draft.

Sources

The SRG ICS-UPLB "Research Proposal" series by Joseph Anthony C. Hermocilla:

1. Part 1: Finding and Identifying Research Gaps
2. Part 2: Writing the Problem Statement
3. Part 3: Writing the Aim, Objectives, and Methods
4. Part 4: Writing the Review of Related Literature
5. Part 5: Writing the Title
6. Part 6: Evaluation and Validation